

AIM2 Preparation

Week 1 - Day 1: Python Fundamentals

5-Minute Review Notes | Variables, Data Types, Input & Basic Logic

Day 1 Goal

Build the basic Python foundation needed for later data analysis and machine learning: store values, understand data types, work with strings and numbers, accept user input, convert types, format output, and make simple decisions.

1. Variables

```
name = "Krishna"  
age = 28  
score = 92.5
```

A variable is a name that refers to a value. Python determines the data type from the assigned value.

2. Core Data Types

Type	Example	Meaning
str	"Python"	Text
int	28	Whole number
float	92.5	Decimal number
bool	True / False	Logical value
NoneType	None	No value / absence of value

```
type(name)  
type(age)  
type(score)
```

3. Strings

```
text = "Artificial Intelligence"  
text.lower()  
text.upper()  
text.title()  
text.strip()  
len(text)
```

Strings are sequences of characters. String methods return transformed versions of the text.

4. String Indexing and Slicing

```
word = "Python"  
word[0] # P  
word[-1] # n  
word[0:3] # Pyt
```

Indexing selects one character. Slicing selects a range. Python indexing starts at 0, and the stop position in a slice is excluded.

5. Numbers and Arithmetic Operators

```
a = 10  
b = 3
```

```
a + b # addition
a - b # subtraction
a * b # multiplication
a / b # division
a // b # floor division
a % b # remainder
a ** b # exponent
```

6. Boolean Values

```
is_passed = True
is_failed = False
```

Booleans represent two logical states: True or False. Conditions and comparisons produce Boolean results.

7. None

```
result = None
```

None represents the absence of a value. It is different from 0, False, or an empty string.

8. User Input

```
name = input("Enter your name: ")
age = input("Enter your age: ")
```

Important: input() returns a string, even when the user types digits.

9. Type Conversion

```
age = int(input("Enter age: "))
price = float(input("Enter price: "))
text_age = str(28)
```

- int() converts a valid value to an integer.
- float() converts a valid value to a decimal number.
- str() converts a value to text.
- Use type() when you want to inspect the current data type.

10. f-Strings

```
name = "Krishna"
average = 88.5
print(f"{name} has an average of {average}")
```

f-strings let you place variables and expressions directly inside a string using { }. They make formatted output easier to read.

11. Basic Conditional Logic

```
average = 82

if average >= 50:
    print("Pass")
else:
    print("Fail")
```

Python uses indentation to define code blocks. The condition is evaluated as True or False, and the matching block runs.

12. Day 1 Mini-Project - Student Performance Analyzer

The project accepted student information and subject scores, then calculated:

- Total score
- Average score
- Pass/fail result
- Formatted student output

Typical calculation pattern

```
total = math + python + ai
average = total / 3
```

```
if average >= 50:
    result = "Pass"
else:
    result = "Fail"
```

Common Day 1 Reminders

- Python does not require semicolons at the end of normal statements.
- `print()` displays output; an f-string is a way to construct formatted text that can be passed to `print()`.
- `input()` returns str, so numeric input normally needs `int()` or `float()` conversion.
- Indentation is part of Python syntax, especially inside if/else blocks.
- Use meaningful variable names such as `math_score` instead of unclear names.

Quick Syntax Cheat Sheet

```
name = "Krishna"
age = 28
score = 92.5
type(age)

text.lower()
text.upper()
text.strip()
text[0]
text[0:3]

input("Enter value: ")
int(value)
float(value)
str(value)

print(f"Name: {name}")

if condition:
    ...
else:
    ...
```

Key Takeaways

- Variables store references to values.
- Data types determine how values behave in operations.
- Strings support indexing, slicing, and useful methods.
- `input()` gives text; convert it when numerical calculations are required.
- f-strings provide clean formatted output.
- Booleans and conditions allow programs to make decisions.
- These fundamentals are the base for every later Python, Pandas, and ML task.

Progress

Week 1 Day 1 - Python Fundamentals: COMPLETE

Mini-project: Student Performance Analyzer.